COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION: CPE21S2**	
DATE PERFORMED:	7/10/2026	**DATE SUBMITTED:**	7/22/26
NAME: ESTORCO, CLINT HOVEN A.		**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA - 2.1 **HOA-ACT 2 ARRAYS, POINTERS, DYNAMIC MEMORY ALLOC**

1. PROCEDURE OUTPUT

TABLE 2-1 INITIAL DRIVER

INPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
 std::string studentName;
 int studentAge;
public:
 //constructor
 Student(std::string newName = "Clint", int newAge = 18){
    studentName =  std::move(newName);
    studentAge = newAge;
    std::cout <<"Constructor called." << std::endl;
 };
 //Deconstructor
 ~Student(){
    std::cout <<"Destructor Called. " << std::endl;
 }
 //Copy COnstructor
Student(const Student &copyStudent){
    std::cout <<"Copy Constructor called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this-> studentAge << std::endl;
}
};

int main(){
```

OUTPUT

```
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2> c
 Constructor called.
Copy Constructor called
 Constructor called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2>
```

```
int main(){

 Student student1("Hugo", 19);
 Student student2(student1);
 Student student3;
 student3 = student2;



    return 0;
}

```

OBSERVATION:
On the code, student1 will get initialized first. Using std::move, the string 'Hugo' is moved to student1's stack memory, the age is stored there, and the message 'Constructor Called' is printed. Student1's destructor is called when the program calls student2 and its data is transferred/overwritten by student2 as well as its age.
Now, the student3 object is created, which declares the constructor again, allocating a new memory address. The value 'Clint' is pushed onto the stack, the value '18' is pushed onto the stack and the message 'Constructor Called' is printed. But in the line student3 = student2, the operator is the assignment '='. The program will lookup their addresses, and student3 address data will be replaced by student2 address data. Lastly, when the program winds up, the destructors will be called back to delete the rest of the stack's objects and print 'Destructor Called'.

TABLE 2-2 MODIFIED DRIVER PROGRAM WITH STUDENT LISTS	
INPUT	**OUTPUT**

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
 std::string studentName;
 int studentAge;
public:
 //constructor
 Student(std::string newName = "Clint", int newAge = 18){
    studentName = newName;
    studentAge = newAge;
    std::cout <<"Constructor called." << std::endl;
 };
 //Deconstructor
 ~Student(){
    std::cout <<"Destructor Called. " << std::endl;
 }
 //Copy COnstructor
Student(const Student &copyStudent){
    std::cout <<"Copy Constructor called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this-> studentAge << std::endl;
}
};

int main(){
    const size_t j = 5;
```

```cpp
    Student studentList[j] = {};
    std::string nameList[j] =  {"Burnice", "Bethoven", "Hoven", "Arsenal", "Alisa"};
    int ageList [j] = {18, 46, 19, 45, 22};


    return 0;
}
```

```
Constructor called.
Copy Constructor called
Constructor called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2>
```

OBSERVATION:

Here's my observation of this C++ code: This code creates a class named Student, which seems to have a constructor, destructor, and copy constructor all necessary for managing the lifecycle of an object. In the class you have a method printDetails, which should print a student's name and age. The main() keeps two constants j and n, with the first assigned the size of the student array, and the second the name array, which in turn is assigned the size of the list of names.The code in the main() function sets up a constant j and a constant n; one (j) for the size of the array of students, and the other (n) for the size of the array of names. These arrays are initialized in the program, but they do not currently contain the appropriate logic to assign the names and ages as appropriate on the Student objects. That means the code as it is doesn't output or use the data that the student has stored in the arrays.

LOOP A- FINAL DRIVER PROGRAM	
INPUT	**OUTPUT**

COMPUTER ENGINEERING

QUEZON CITY

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
 std::string studentName;
 int studentAge;
public:
 //constructor
 Student(std::string newName = "Clint", int newAge = 18){
    studentName = newName;
    studentAge = newAge;
    std::cout <<"Constructor called." << std::endl;
 };
 //Deconstructor
 ~Student(){
    std::cout <<"Destructor Called. " << std::endl;
 }
 //Copy COnstructor
Student(const Student &copyStudent){
    std::cout <<"Copy Constructor called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this-> studentAge << std::endl;
}
};

int main(){
    const size_t j = 5;
```

```cpp
    Student studentList[j] = {};
    std::string nameList[j] =  {"Burnice", "Bethoven", "Hoven", "Arsenal", "Alisa"};
    int ageList [j] = {18, 46, 19, 45, 22};

    for (int i = 0; i < j; i++){ //Loop A
    Student *ptr = new Student(nameList[i], ageList[i]);
    studentList[i] = *ptr;
    }


    return 0;
}
```

```
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2> cd "c:\Users\TI
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
```

```
Constructor called.
Constructor called.
Constructor called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2>
```

OBSERVATION:

In my observation inLoop A uses a 'for' loop to loop for the given size of the defined value of 'j'. On every iteration dynamically allocates a new Student object on the heap. The constructor is called for every object, with the values from the nameList and ageList arrays. These newly created pointers are then put into the respective index in the studentList array. During this process, we create 5 separate Student objects, giving them their attributes.

LOOP B- FINAL DRIVER PROGRAM	
INPUT	**OUTPUT**

COMPUTER ENGINEERING

QUEZON CITY

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
 std::string studentName;
 int studentAge;
public:
 //constructor
 Student(std::string newName = "Clint", int newAge = 18){
    studentName = newName;
    studentAge = newAge;
    std::cout <<"Constructor called." << std::endl;
 };
 //Deconstructor
 ~Student(){
    std::cout <<"Destructor Called. " << std::endl;
 }
 //Copy COnstructor
Student(const Student &copyStudent){
    std::cout <<"Copy Constructor called" << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails(){
    std::cout << this->studentName << " " << this-> studentAge << std::endl;
}
};

int main(){
    const size_t j = 5;
```

```cpp
    Student studentList[j] = {};
    std::string nameList[j] =  {"Burnice", "Bethoven", "Hoven", "Arsenal", "Alisa"};
    int ageList [j] = {18, 46, 19, 45, 22};

    for (int i = 0; i < j; i++){//Loop B
    studentList[i].printDetails();
    }
    return 0;
}
```

```
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2> cd "c:\Users\TIPQC\Desktop\ESTORCO_CPE21
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Constructor called.
Clint 18
Clint 18
Clint 18
Clint 18
```

```
Clint 18
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Desktop\ESTORCO_CPE21S2_ACT2>
```

OBSERVATION:

In my observation, this code contains a class named Student that has two class variables: a string variable name, which is set to the default value of "Clint", and an integer variable age, which is set to the default value of 18. In main(), 5 Student objects are created by default, meaning that the constructor is called 5 times, and Constructor called. is printed 5 times. A loop is then invoked which calls printDetails() on each object, which prints Clint 18, since no objects have their arrays assigned to them. Finally, the destructor runs five times by itself when the program ends, to clean up each object safely from memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

INPUT	OUTPUT

Supplementary_Act.cpp

```cpp
#include <iostream>
#include <string>

class Item {
private:
    std::string* Name;
    int* Price;
    int* Quantity;

public:
    // Constructor
    Item(std::string newName = "Place_Holder", int newPrice = 5, int newQuantity = 5) {
        Name = new std::string(newName);
        Price = new int(newPrice);
        Quantity = new int(newQuantity);

        std::cout << "Constructor Successfully Called.\n";
    }

    // Destructor
    ~Item() {
        delete Name;
        delete Price;
        delete Quantity;

        std::cout << "Destructor Successfully Called.\n";
    }

    // Copy Constructor
    Item(const Item& other) {
        Name = new std::string(*other.Name);
        Price = new int(*other.Price);
        Quantity = new int(*other.Quantity);

        std::cout << "Copy Constructor Successfully Called.\n";
    }

    // Copy Assignment Operator
    Item& operator=(const Item& other) {
        if (this != &other) {
            delete Name;
            delete Price;
            delete Quantity;

            Name = new std::string(*other.Name);
            Price = new int(*other.Price);
            Quantity = new int(*other.Quantity);
        }

        std::cout << "Copy Assignment Operator Successfully Called.\n";
        return *this;
    }

    void display() {
        std::cout << *Name << " PHP" << *Price << " x" << *Quantity << std::endl;
    }

    double totalSum() {
        return (*Price) * (*Quantity);
    }
};

int main() {

    Item* ShoppingList[4];

    ShoppingList[0] = new Item("Apple", 10, 7);
    ShoppingList[1] = new Item("Banana", 10, 8);
    ShoppingList[2] = new Item("Broccoli", 60, 12);
    ShoppingList[3] = new Item("Lettuce", 50, 10);

    double total = 0;

    for (int i = 0; i < 4; i++) {
        ShoppingList[i]->display();
        std::cout << "Subtotal: " << ShoppingList[i]->totalSum() << std::endl;
        total += ShoppingList[i]->totalSum();
    }

    std::cout << "\nTotal of all items: " << total << std::endl;

    // Delete all objects
    for (int i = 0; i < 4; i++) {
        delete ShoppingList[i];
        ShoppingList[i] = NULL;
    }

    return 0;
}
```

```
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Constructor Successfully Called.
Apple PHP10 x7
Subtotal: 70
Banana PHP10 x8
Subtotal: 80
Broccoli PHP60 x12
Subtotal: 720
Lettuce PHP50 x10
Subtotal: 500

Total of all items: 1370
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.
Destructor Successfully Called.

--------------------------------
Process exited after 0.02293 seconds with return value 0
Press any key to continue . . .
```

CONCLUSION:

This lab activity examined C++ classes, object initialization, dynamic allocation and manipulation of pointers. By performing the step-by-step code for object drivers and array implementations, the exact life cycles of constructors and copy assignment operators as well as destructors were studied. In conclusion, the exercise helped participants understand the concept of dynamic allocation, learn how to implement it effectively, and understand the proper disposal methods such as the delete operator for a robust memory management system in software development.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly (paraphrasing)**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and paraphrasing**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for fixing grammar and punctuation**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.